UQA Python API Reference

This document covers the Python API for UQA (Unified Query Algebra), including the Engine, the fluent QueryBuilder API, core types, text analysis, storage, graph operations, and scoring modules.

UQA Python API Reference

- 1. Engine
  - Constructor
  - SQL Interface
  - Fluent QueryBuilder
  - Document Management
  - Graph Management
  - Named Graphs
  - Analyzer Management
  - Scoring Parameters
  - Transactions
  - Context Manager
- 2. SQLResult
  - Properties
  - Methods
  - Example
- 3. QueryBuilder (Fluent API)
  - Term Retrieval
  - Vector Search
  - Boolean Algebra
  - Filtering
  - Scoring
  - Fusion
  - Graph Operations
  - Vector Exclusion
  - Joins
  - Aggregation
  - Hierarchical Data
  - Execution
- 4. Core Types
  - PostingEntry
  - Payload
  - GeneralizedPostingEntry
  - Vertex
  - Edge
  - IndexStats
  - Predicates
- 5. PostingList
  - Boolean Algebra
  - Methods
  - Iteration
- 6. Text Analysis
  - Analyzer
  - Built-in Analyzers
  - Tokenizers
  - Token Filters
  - Character Filters

- Analyzer Registry
- Serialization

7. Storage Layer
    - DocumentStore
    - InvertedIndex
    - SpatialIndex (R*Tree)
    - HNSWIndex (Vector Index)
8. Graph Store
    - Graph Indexes
    - Graph Patterns
9. Scoring
    - BM25
    - Bayesian BM25
    - Vector Scoring
10. Query Planning
    - Optimizer
    - Cost Model
    - Cardinality Estimator
    - Join Order Optimizer (DPccp)
    - Plan Executor
11. Operators
    - Primitive Operators
    - Boolean Operators
    - Graph Operators
    - Hierarchical Operators
    - Join Operators
    - Execution Context
12. Transaction
    - Methods
    - Properties
    - Usage
13. Table Schema
    - ColumnDef
    - ColumnStats
    - ForeignKeyDef
14. Complete Example

---

1. Engine

The `Engine` class is the main entry point for all UQA operations.

```
from uqa.engine import Engine
```

Constructor

```
Engine(
    db_path: str | None = None,
    parallel_workers: int = 4,
    spill_threshold: int = 0,
)
```

Parameter	Description
`db_path`	Path to SQLite file for persistence. `None` for in-memory.
`parallel_workers`	Number of parallel execution workers.
`spill_threshold`	Memory threshold (bytes) for disk spilling. 0 disables.

SQL Interface

```
engine.sql(query: str, params: list[Any] | None = None) -> SQLResult
```

Execute a SQL query. Returns an `SQLResult` object with `.columns` and `.rows`. Supports parameterized queries with `$1`, `$2`, etc.

```
result = engine.sql("SELECT * FROM users WHERE age > $1", [25])
for row in result.rows:
    print(row["name"], row["age"])
```

Fluent QueryBuilder

```
engine.query(table: str) -> QueryBuilder
```

Create a `QueryBuilder` scoped to a table for fluent API queries.

```
results = (
    engine.query(table="articles")
    .term("attention", field="body")
    .score_bm25("attention")
    .execute()
)
```

Document Management

```
engine.add_document(
    doc_id: DocId,
    document: dict[str, Any],
    table: str,
    embedding: NDArray | None = None,
) -> None
```

Add a document to a table's storage and indexes.

```
engine.get_document(doc_id: DocId, table: str) -> dict[str, Any] | None
```

Retrieve a document by its ID from the given table. Returns the stored document dict, or `None` if the document does not exist.

```
engine.delete_document(doc_id: DocId, table: str) -> None
```

Remove a document from a table's storage and indexes.

Graph Management

```
engine.add_graph_vertex(vertex: Vertex, table: str) -> None
engine.add_graph_edge(edge: Edge, table: str) -> None
engine.get_graph_store(table: str) -> GraphStore
```

Add vertices and edges to a table's graph store. `get_graph_store()` returns the `GraphStore` associated with a table for direct vertex lookups and index building.

Named Graphs

```
engine.create_graph(name: str) -> GraphStore
engine.drop_graph(name: str) -> None
engine.get_graph(name: str) -> GraphStore
engine.has_graph(name: str) -> bool
```

Manage named graphs for Cypher queries. Named graphs are persisted via the SQLite catalog.

Analyzer Management

```
engine.create_analyzer(name: str, config: dict[str, Any]) -> None
engine.drop_analyzer(name: str) -> None
engine.set_table_analyzer(table_name: str, field: str, analyzer_name: str, phase: str = "both") -> None
engine.get_table_analyzer(table_name: str, field: str, phase: str = "index") -> Any
```

Create named analyzers and assign them to table fields for custom tokenization.

The `phase` parameter in `set_table_analyzer` controls when the analyzer is applied:

- `"index"` — used during document indexing (controls what tokens are stored)
- `"search"` — used during query processing (controls how query terms are expanded)
- `"both"` — assigned to both phases (default)

The `phase` parameter in `get_table_analyzer` selects which phase's analyzer to retrieve (`"index"` by default). The search-time analyzer falls back to the index-time analyzer, which falls back to the default analyzer.

```
# Dual analyzer: index without synonyms, search with synonym expansion
engine.set_table_analyzer("products", "body", "plain_analyzer", phase="index")
engine.set_table_analyzer("products", "body", "synonym_analyzer", phase="search")
```

Scoring Parameters

```python
engine.save_scoring_params(name: str, params: dict[str, Any]) -> None
engine.load_scoring_params(name: str) -> dict[str, Any] | None
engine.load_all_scoring_params() -> list[tuple[str, dict[str, Any]]]
```

Persist and retrieve Bayesian calibration parameters for named scoring signals.

Transactions

```python
engine.begin() -> Transaction
```

Start an explicit transaction. Returns a `Transaction` context manager.

```python
with engine.begin() as tx:
    engine.sql("INSERT INTO t VALUES (1, 'a')")
    engine.sql("INSERT INTO t VALUES (2, 'b')")
    # auto-commits on exit; auto-rollbacks on exception
```

Context Manager

```python
with Engine(db_path="data.db") as engine:
    engine.sql("SELECT 1")
# engine.close() called automatically
```

---

2. SQLResult

Returned by `engine.sql()`.

```python
from uqa.sql.compiler import SQLResult
```

Properties

Property	Type	Description
`columns`	`list[str]`	Column names
`rows`	`list[dict[str, Any]]`	Row data as list of dicts

Methods

```python
result.to_arrow() -> pa.Table       # Convert to PyArrow Table
result.to_parquet(path: str) -> None # Write to Parquet file
len(result)                          # Number of rows
str(result)                          # Formatted table output
```

Example

```python
result = engine.sql("SELECT id, name FROM users ORDER BY id LIMIT 5")
print(result.columns)  # ['id', 'name']
for row in result.rows:
    print(row["id"], row["name"])

# Export to Arrow/Parquet
arrow_table = result.to_arrow()
result.to_parquet("users.parquet")
```

---

3. QueryBuilder (Fluent API)

The `QueryBuilder` provides a chainable, type-safe interface for building queries across all paradigms.

```python
from uqa.api.query_builder import QueryBuilder
```

All methods return a new `QueryBuilder`, enabling method chaining.

Term Retrieval

```python
.term(term: str, field: str | None = None) -> QueryBuilder
```

Retrieve documents containing a term. The query term is automatically analyzed using the same analyzer that indexed the field.

```python
results = engine.query("articles").term("attention", field="body").execute()
```

Vector Search

```python
.vector(query: NDArray, threshold: float, field: str = "embedding") -> QueryBuilder
.knn(query: NDArray, k: int, field: str = "embedding") -> QueryBuilder
```

Method	Description
`vector()`	All documents with similarity >= threshold
`knn()`	Top-k nearest neighbors

```python
import numpy as np
q = np.random.randn(64).astype(np.float32)
results = engine.query("papers").knn(q, k=10).execute()
```

Boolean Algebra

```
.and_(other: QueryBuilder) -> QueryBuilder   # Intersection
.or_(other: QueryBuilder) -> QueryBuilder    # Union
.not_() -> QueryBuilder                      # Complement
```

```
text_q = engine.query("products").term("wireless")
vec_q = engine.query("products").knn(audio_vec, k=10)
combined = text_q.and_(vec_q)
results = combined.execute()
```

Filtering

```
.filter(field: str, predicate: Predicate) -> QueryBuilder
```

Filter results by a field predicate. See Predicates for all available predicate types.

```
from uqa.core.types import GreaterThanOrEqual, InSet

results = (
    engine.query("products")
    .term("laptop")
    .filter("price", GreaterThanOrEqual(500))
    .filter("category", InSet(frozenset({"electronics", "computers"})))
    .execute()
)
```

Scoring

```
.score_bm25(query: str, field: str | None = None) -> QueryBuilder
.score_bayesian_bm25(query: str, field: str | None = None) -> QueryBuilder
```

Method	Score Range	Description
`score_bm25()`	[0, +inf)	Standard BM25 relevance score
`score_bayesian_bm25()`	[0, 1]	Calibrated probability of relevance

```
results = (
    engine.query("articles")
    .term("transformer", field="title")
    .score_bm25("transformer")
    .execute()
)
for entry in results:
    print(f"doc={entry.doc_id}, score={entry.payload.score:.4f}")
```

Fusion

```
.fuse_log_odds(*builders: QueryBuilder, alpha: float = 0.5) -> QueryBuilder
.fuse_prob_and(*builders: QueryBuilder) -> QueryBuilder
.fuse_prob_or(*builders: QueryBuilder) -> QueryBuilder
```

Combine multiple relevance signals with calibrated probability fusion.

```
text_q = engine.query("papers").term("attention").score_bayesian_bm25("attention")
vec_q = engine.query("papers").knn(query_vec, k=20)

# Log-odds fusion
fused = engine.query("papers").fuse_log_odds(text_q, vec_q)
results = fused.execute()
```

Graph Operations

```
.traverse(start: int, label: str | None = None, max_hops: int = 1) -> QueryBuilder
.match_pattern(pattern: GraphPattern) -> QueryBuilder
.rpq(expr: str, start: int | None = None) -> QueryBuilder
.vertex_aggregate(property_name: str, agg_fn: str = "sum") -> AggregateResult
```

```
# BFS traversal: 3-hop from vertex 1 along 'manages' edges
team = engine.query("org").traverse(1, "manages", max_hops=3).execute()

# Aggregate vertex properties
salary = engine.query("org").traverse(1, "manages", 3).vertex_aggregate("salary", "sum")
print(f"Total salary: ${salary.value:,}")

# Regular path query with Kleene star
reachable = engine.query("org").rpq("manages*", start=1).execute()
```

Vector Exclusion

```
.vector_exclude(negative_vector: NDArray, threshold: float) -> QueryBuilder
```

Exclude documents similar to a negative query vector.

Joins

```
.join(other: QueryBuilder, left_field: str, right_field: str) -> QueryBuilder
.vector_join(other: QueryBuilder, left_field: str, right_field: str, threshold: float) -> QueryBuilder
```

Aggregation

```
.aggregate(field: str, agg: str) -> AggregateResult
.facet(field: str) -> FacetResult
.vector_facet(field: str, query_vector: NDArray, threshold: float) -> FacetResult
```

Method	Returns	Description
`aggregate()`	`AggregateResult`	Collection-level aggregation (count, sum, avg, min, max)
`facet()`	`FacetResult`	Facet counts by field value
`vector_facet()`	`FacetResult`	Facet counts conditioned on vector similarity

```
# Per-collection aggregation
avg_price = engine.query("products").knn(q, 10).aggregate("price", "avg")
print(f"Average price: ${avg_price.value:.2f}")

# Facet counts
facets = engine.query("products").term("wireless").facet("category")
for value, count in facets.counts.items():
    print(f"  {value}: {count}")
```

Hierarchical Data

```
.path_filter(path: PathExpr, predicate: Predicate) -> QueryBuilder
.path_project(*paths: PathExpr) -> QueryBuilder
.unnest(path: PathExpr) -> QueryBuilder
.path_aggregate(path: str | PathExpr, agg: str) -> QueryBuilder
```

Operate on nested JSON-like documents.

```
from uqa.core.types import Equals, GreaterThan

# Filter by nested field
results = (
    engine.query("orders")
    .path_filter(["shipping", "city"], Equals("Seoul"))
    .execute()
)

# Aggregate nested array values
totals = (
    engine.query("orders")
    .path_aggregate("items.price", "sum")
    .execute()
)
```

Execution

```
.execute() -> PostingList                    # Execute and return PostingList
.execute_arrow() -> pa.Table                 # Execute and return PyArrow Table
.execute_parquet(path: str) -> None          # Execute and write to Parquet
.explain() -> str                            # Return optimized query plan
```

```
# Arrow export
table = engine.query("articles").term("attention").execute_arrow()
print(table.column_names)  # ['_doc_id', '_score']
print(table.num_rows)

# Parquet export
engine.query("articles").term("attention").execute_parquet("results.parquet")

# Explain plan
plan = engine.query("articles").term("attention").score_bm25("attention").explain()
print(plan)
```

---

4. Core Types

```
from uqa.core.types import (
    PostingEntry, Payload, GeneralizedPostingEntry,
    Vertex, Edge, IndexStats,
    Equals, NotEquals, GreaterThan, GreaterThanOrEqual,
    LessThan, LessThanOrEqual, Between, InSet,
    IsNull, IsNotNull, Like, NotLike, ILike, NotILike,
)
```

PostingEntry

A single entry in a posting list.

```
@dataclass(frozen=True)
class PostingEntry:
    doc_id: int
    payload: Payload
```

Payload

Carries positions, score, and field values.

```
@dataclass(frozen=True)
class Payload:
    positions: tuple[int, ...] = ()
    score: float = 0.0
    fields: dict[str, Any] = field(default_factory=dict)
```

GeneralizedPostingEntry

Join result entry carrying a tuple of document IDs.

```python
@dataclass(frozen=True)
class GeneralizedPostingEntry:
    doc_ids: tuple[int, ...]
    payload: Payload
```

Vertex

```python
@dataclass(frozen=True)
class Vertex:
    vertex_id: int
    label: str
    properties: dict[str, Any] = field(default_factory=dict)
```

Edge

```python
@dataclass(frozen=True)
class Edge:
    edge_id: int
    source_id: int
    target_id: int
    label: str
    properties: dict[str, Any] = field(default_factory=dict)
```

IndexStats

```python
@dataclass
class IndexStats:
    total_docs: int = 0
    avg_doc_length: float = 0.0
    dimensions: int = 0
```

Predicates

All predicates implement `evaluate(value: Any) -> bool`.

Predicate	Constructor	Matches when
`Equals(target)`	`Equals(42)`	`value == target`
`NotEquals(target)`	`NotEquals(42)`	`value != target`
`GreaterThan(target)`	`GreaterThan(10)`	`value > target`
`GreaterThanOrEqual(target)`	`GreaterThanOrEqual(10)`	`value >= target`
`LessThan(target)`	`LessThan(100)`	`value < target`
`LessThanOrEqual(target)`	`LessThanOrEqual(100)`	`value <= target`
`Between(low, high)`	`Between(10, 100)`	`low <= value <= high`
`InSet(values)`	`InSet(frozenset({1,2,3}))`	`value in values`
`IsNull()`	`IsNull()`	`value is None`
`IsNotNull()`	`IsNotNull()`	`value is not None`
`Like(pattern)`	`Like("%hello%")`	SQL LIKE (case-sensitive)
`NotLike(pattern)`	`NotLike("%test%")`	SQL NOT LIKE
`ILike(pattern)`	`ILike("%hello%")`	SQL ILIKE (case-insensitive)
`NotILike(pattern)`	`NotILike("%test%")`	SQL NOT ILIKE

---

5. PostingList

The universal data structure for query results.

```
from uqa.core.posting_list import PostingList, GeneralizedPostingList
```

Boolean Algebra

```
pl1.union(pl2) -> PostingList        # A | B
pl1.intersect(pl2) -> PostingList    # A & B
pl1.difference(pl2) -> PostingList   # A - B
pl1.complement(universal) -> PostingList  # U - A
```

Operator overloads: `|` (union), `&` (intersect), `-` (difference).

Methods

Method	Description
`get_entry(doc_id)`	Retrieve entry by doc_id (binary search)
`top_k(k)`	Return top-k entries by score
`with_scores(score_fn)`	Apply scoring function to all entries
`doc_ids`	Property: set of all document IDs
`entries`	Property: list of PostingEntry

Iteration

```python
results = engine.query("articles").term("attention").execute()
for entry in results:
    print(f"doc_id={entry.doc_id}, score={entry.payload.score:.4f}")
    print(f"fields={entry.payload.fields}")
```

---

6. Text Analysis

```python
from uqa.analysis.analyzer import (
    Analyzer, DEFAULT_ANALYZER,
    standard_analyzer, whitespace_analyzer, keyword_analyzer,
    register_analyzer, get_analyzer, drop_analyzer, list_analyzers,
)
```

Analyzer

```python
Analyzer(
    tokenizer: Tokenizer | None = None,
    token_filters: list[TokenFilter] | None = None,
    char_filters: list[CharFilter] | None = None,
)
```

```python
analyzer = Analyzer(
    tokenizer=StandardTokenizer(),
    token_filters=[LowerCaseFilter(), PorterStemFilter()],
    char_filters=[HTMLStripCharFilter()],
)
tokens = analyzer.analyze("The quick brown fox")  # ['quick', 'brown', 'fox']
```

Built-in Analyzers

Four built-in analyzers are registered by default under names `"whitespace"`, `"standard"`, `"standard_cjk"`, and `"keyword"`. They cannot be overwritten or dropped.

Name	Factory	Pipeline
`"standard"`	`standard_analyzer(language="english")`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter -> PorterStemFilter
`"whitespace"`	`whitespace_analyzer()`	WhitespaceTokenizer -> LowerCaseFilter
`"standard_cjk"`	`standard_cjk_analyzer(language="english")`	StandardTokenizer -> LowerCaseFilter -> ASCIIFoldingFilter -> StopWordFilter -> PorterStemFilter -> NGramFilter(2, 3, keep_short=True)
`"keyword"`	`keyword_analyzer()`	KeywordTokenizer (no filters)

`DEFAULT_ANALYZER` is `standard_analyzer()`. When no analyzer is assigned to a field, the default analyzer is used for both indexing and searching.

Tokenizers

```python
from uqa.analysis.tokenizer import (
    WhitespaceTokenizer, StandardTokenizer, LetterTokenizer,
    NGramTokenizer, PatternTokenizer, KeywordTokenizer,
)
```

Class	Description	Parameters
`WhitespaceTokenizer`	Split on whitespace (`str.split()`)	(none)
`StandardTokenizer`	Unicode word-boundary tokenizer (`\w+` regex)	(none)
`LetterTokenizer`	Extract runs of ASCII letters only (`[a-zA-Z]+`)	(none)
`NGramTokenizer(min_gram, max_gram)`	Split on whitespace, then generate character n-grams per word	`min_gram` (default 1), `max_gram` (default 2)
`PatternTokenizer(pattern)`	Split text using a regex delimiter	`pattern` (default `r"\W+"`)
`KeywordTokenizer`	Entire input as a single token, unmodified	(none)

Token Filters

```python
from uqa.analysis.token_filter import (
    LowerCaseFilter, StopWordFilter, PorterStemFilter,
    ASCIIFoldingFilter, SynonymFilter, NGramFilter,
    EdgeNGramFilter, LengthFilter,
)
```

Class	Description	Parameters
`LowerCaseFilter`	Convert all tokens to lowercase	(none)
`StopWordFilter(language, custom_words)`	Remove stop words from the token stream	`language` (default `"english"`), `custom_words` (optional `set[str]`)
`PorterStemFilter`	Apply the Porter stemming algorithm (suffix stripping)	(none)
`ASCIIFoldingFilter`	Fold Unicode characters to ASCII equivalents (NFKD normalization)	(none)
`SynonymFilter(synonyms, synonyms_path)`	Expand tokens with synonyms; original token kept, alternatives appended	`synonyms`: `dict[str, list[str]]` or `synonyms_path`: file path (mutually exclusive)
`NGramFilter(min_gram, max_gram, keep_short)`	Generate character n-grams from each token	`min_gram` (default 2), `max_gram` (default 3), `keep_short` (default False)
`EdgeNGramFilter(min_gram, max_gram)`	Generate prefix n-grams (autocomplete/typeahead)	`min_gram` (default 1), `max_gram` (default 20)
`LengthFilter(min_length, max_length)`	Keep tokens within a length range	`min_length` (default 0), `max_length` (default 0 = no limit)

Character Filters

```
from uqa.analysis.char_filter import (
    HTMLStripCharFilter, MappingCharFilter, PatternReplaceCharFilter,
)
```

Class	Description	Parameters
`HTMLStripCharFilter`	Strip HTML tags and convert common HTML entities	(none)
`MappingCharFilter(mapping)`	Apply string-level character replacements (longest-match-first)	`mapping`: `dict[str, str]`
`PatternReplaceCharFilter(pattern, replacement)`	Replace text matching a regular expression	`pattern`: regex string, `replacement` (default `""`)

Analyzer Registry

```
register_analyzer("my_analyzer", analyzer)    # Register globally
get_analyzer("my_analyzer")                   # Retrieve by name
drop_analyzer("my_analyzer")                  # Remove
list_analyzers()                              # List all names
```

Serialization

```
config = analyzer.to_dict()    # dict
json_str = analyzer.to_json()  # JSON string

restored = Analyzer.from_dict(config)
restored = Analyzer.from_json(json_str)
```

---

7. Storage Layer

DocumentStore

```
from uqa.storage.document_store import DocumentStore

store = DocumentStore()
store.put(1, {"name": "Alice", "age": 30})
doc = store.get(1)            # {"name": "Alice", "age": 30}
store.get_field(1, "name")    # "Alice"
store.delete(1)
len(store)                    # number of documents
store.doc_ids                 # set of all doc IDs
```

InvertedIndex

```python
from uqa.storage.inverted_index import InvertedIndex

idx = InvertedIndex(analyzer=standard_analyzer())
indexed_terms = idx.add_document(1, {"title": "hello world", "body": "..."})
pl = idx.get_posting_list("title", "hello")    # PostingList for field+term
pl = idx.get_posting_list_any_field("hello")   # PostingList across all fields
freq = idx.doc_freq("title", "hello")          # document frequency
idx.remove_document(1)
```

`InvertedIndex` supports per-field analyzer overrides:

```python
idx.set_field_analyzer("body", custom_analyzer)
analyzer = idx.get_field_analyzer("body")
```

SpatialIndex (R*Tree)

Spatial columns (`POINT`) store `[longitude, latitude]` pairs in the document store. R*Tree indexes must be created explicitly via `CREATE INDEX ... USING rtree`. Without an index, `spatial_within()` falls back to brute-force Haversine scan.

```cpp
-- Define a POINT column (storage only, no index)
CREATE TABLE places (id SERIAL PRIMARY KEY, location POINT);

-- Create an R*Tree index explicitly
CREATE INDEX idx_loc ON places USING rtree (location);

-- Insert with POINT constructor
INSERT INTO places (location) VALUES (POINT(-73.9857, 40.7484));
```

Direct Python API:

```python
from uqa.storage.spatial_index import SpatialIndex, haversine_distance

index = SpatialIndex("places", "location")
index.add(1, -73.9857, 40.7484)  # (doc_id, longitude, latitude)

# Range search: all points within 2km
results = index.search_within(-73.9973, 40.7308, 2000)  # returns PostingList

# Haversine distance
dist = haversine_distance(40.7308, -73.9973, 40.7484, -73.9857)  # meters
```

HNSWIndex (Vector Index)

Vector columns (`VECTOR(n)`) store data in the document store. HNSW indexes must be created explicitly via `CREATE INDEX ... USING hnsw`. Without an index, `knn_match()` falls back to brute-force exact cosine similarity scan.

```sql
-- Define a vector column (storage only, no index)
CREATE TABLE docs (id SERIAL PRIMARY KEY, emb VECTOR(384));

-- Create an HNSW index explicitly
CREATE INDEX idx_emb ON docs USING hnsw (emb);

-- With custom parameters
CREATE INDEX idx_emb ON docs USING hnsw (emb) WITH (ef_construction = 200, m = 16);
```

Direct Python API:

```python
from uqa.storage.vector_index import HNSWIndex
import numpy as np

index = HNSWIndex(dimensions=64, ef_construction=200, m=16)
index.add(1, np.random.randn(64).astype(np.float32))

# K-nearest neighbors
results = index.search_knn(query_vec, k=10)

# Threshold search
results = index.search_threshold(query_vec, threshold=0.8)
```

The index auto-resizes as elements are added — no capacity parameter is needed.

---

8. Graph Store

```python
from uqa.graph.store import GraphStore
from uqa.core.types import Vertex, Edge

gs = GraphStore()

# Add vertices
gs.add_vertex(Vertex(1, "Person", {"name": "Alice"}))
gs.add_vertex(Vertex(2, "Person", {"name": "Bob"}))

# Add edges
gs.add_edge(Edge(1, source_id=1, target_id=2, label="KNOWS", properties={}))

# Query
v = gs.get_vertex(1)
neighbors = gs.neighbors(1, label="KNOWS", direction="out")  # [2]
people = gs.vertices_by_label("Person")  # [Vertex(1, ...), Vertex(2, ...)]

# Remove
gs.remove_vertex(1)  # also removes incident edges
gs.remove_edge(1)
```

Graph Indexes

```python
from uqa.graph.index import LabelIndex, NeighborhoodIndex, PathIndex

# Label index: fast lookup of edges by label
label_idx = LabelIndex.build(gs)
edge_ids = label_idx.edges_by_label("KNOWS")

# Neighborhood index: pre-computed k-hop neighborhoods
nbr_idx = NeighborhoodIndex.build(gs, max_hops=3)
reachable = nbr_idx.neighbors(vertex_id=1, hops=2)

# Path index: pre-computed labeled paths
path_idx = PathIndex.build(gs, label_sequences=[["KNOWS", "WORKS_AT"]])
pairs = path_idx.lookup(["KNOWS", "WORKS_AT"])  # [(source, target), ...]
```

Graph Patterns

```python
from uqa.graph.pattern import VertexPattern, EdgePattern, GraphPattern

pattern = GraphPattern(
    vertex_patterns=[
        VertexPattern("a", constraints=[]),
        VertexPattern("b", constraints=[]),
    ],
    edge_patterns=[
        EdgePattern("a", "b", "KNOWS", constraints=[]),
    ],
)
```

---

9. Scoring

BM25

```python
from uqa.scoring.bm25 import BM25Scorer, BM25Params

scorer = BM25Scorer(BM25Params(k1=1.5, b=0.75), index_stats)
score = scorer.score(term_freq=3, doc_length=100, doc_freq=50)
idf = scorer.idf(doc_freq=50)
combined = scorer.combine_scores([score1, score2])
```

Bayesian BM25

```python
from uqa.scoring.bayesian_bm25 import BayesianBM25Scorer, BayesianBM25Params

scorer = BayesianBM25Scorer(
    BayesianBM25Params(k1=1.5, b=0.75, alpha=0.5, beta=0.5, base_rate=0.1),
    index_stats,
)
prob = scorer.score(term_freq=3, doc_length=100, doc_freq=50)  # P(rel) in [0,1]
```

Vector Scoring

```
from uqa.scoring.vector import VectorScorer

sim = VectorScorer.cosine_similarity(vec_a, vec_b)
prob = VectorScorer.similarity_to_probability(sim)  # map [-1,1] to [0,1]
```

---

10. Query Planning

Optimizer

```
from uqa.planner.optimizer import QueryOptimizer

optimizer = QueryOptimizer(index_stats)
optimized_op = optimizer.optimize(operator_tree)
```

Optimization passes:

- Filter pushdown (into scans, intersect children, graph patterns)
- Vector threshold merge
- Intersect reordering by cardinality

Cost Model

```
from uqa.planner.cost_model import CostModel

model = CostModel()
cost = model.estimate(operator, index_stats)
```

Cardinality Estimator

```
from uqa.planner.cardinality import CardinalityEstimator

estimator = CardinalityEstimator()
card = estimator.estimate(operator, index_stats)
```

Join Order Optimizer (DPccp)

```
from uqa.planner.join_order import JoinOrderOptimizer

optimizer = JoinOrderOptimizer()
join_op, table = optimizer.optimize(relations, predicates)
```

The DPccp algorithm enumerates connected subgraph complement pairs to find the optimal join order. For small cardinalities (<= 100), `IndexJoinOperator` (binary search) is selected; otherwise `InnerJoinOperator` (hash join with build-on-smaller-side) is used.

Plan Executor

```python
from uqa.planner.executor import PlanExecutor

executor = PlanExecutor(execution_context)
result = executor.execute(optimized_op)
plan_str = executor.explain(optimized_op)
```

---

11. Operators

All operators implement `execute(context: ExecutionContext) -> PostingList`.

Primitive Operators

```python
from uqa.operators.primitive import (
    TermOperator, VectorSimilarityOperator, KNNOperator,
    SpatialWithinOperator, FilterOperator, FacetOperator, ScoreOperator,
)
```

Operator	Description
`TermOperator(term, field)`	Retrieve posting list for a term
`VectorSimilarityOperator(query, threshold, field)`	Vector similarity >= threshold
`KNNOperator(query, k, field)`	Top-k nearest neighbors
`SpatialWithinOperator(field, cx, cy, dist)`	Spatial range query (R*Tree + Haversine)
`FilterOperator(field, predicate, source)`	Filter entries by predicate
`FacetOperator(field, source)`	Compute facet counts
`ScoreOperator(scorer, source, terms)`	Apply scoring function

Boolean Operators

```python
from uqa.operators.boolean import (
    UnionOperator, IntersectOperator, ComplementOperator,
)
```

Operator	Description
`UnionOperator(sources)`	Union of posting lists
`IntersectOperator(sources)`	Intersection of posting lists
`ComplementOperator(source)`	Complement of posting list

Graph Operators

```python
from uqa.graph.operators import (
    TraverseOperator, PatternMatchOperator, RegularPathQueryOperator,
    VertexAggregationOperator,
)
```

Operator	Description
`TraverseOperator(start, label, max_hops)`	BFS graph traversal
`PatternMatchOperator(pattern)`	Subgraph isomorphism
`RegularPathQueryOperator(path_expr, start)`	NFA-based path query
`VertexAggregationOperator(source, property, agg_fn)`	Aggregate vertex properties

Hierarchical Operators

```python
from uqa.operators.hierarchical import (
    PathFilterOperator, PathProjectOperator,
    PathUnnestOperator, PathAggregateOperator,
)
```

Join Operators

```python
from uqa.joins.inner import InnerJoinOperator
from uqa.joins.index import IndexJoinOperator
from uqa.joins.base import JoinCondition

condition = JoinCondition(left_field="id", right_field="fk")
join = InnerJoinOperator(left_source, right_source, condition)
result = join.execute(context)
```

Operator	Complexity	Description
`InnerJoinOperator`	O(N + M)	Hash join, builds on smaller side
`IndexJoinOperator`	O(N log M)	Binary search join

Execution Context

```python
from uqa.operators.base import ExecutionContext

context = ExecutionContext(
    document_store=doc_store,
    inverted_index=inv_idx,
    vector_indexes={"embedding": hnsw_index},
    spatial_indexes={"location": spatial_index},
    graph_store=graph_store,
)
```

---

12. Transaction

```python
from uqa.storage.transaction import Transaction
```

Methods

Method	Description
`commit()`	Commit the transaction
`rollback()`	Rollback the transaction
`savepoint(name)`	Create a savepoint
`release_savepoint(name)`	Release a savepoint
`rollback_to(name)`	Rollback to a savepoint

Properties

Property	Type	Description
`active`	`bool`	Whether transaction is active

Usage

```python
with engine.begin() as tx:
    engine.sql("INSERT INTO accounts (id, balance) VALUES (1, 1000)")
    tx.savepoint("sp1")
    engine.sql("UPDATE accounts SET balance = 500 WHERE id = 1")
    tx.rollback_to("sp1")  # undo the update
    # commits on exit
```

---

13. Table Schema

```python
from uqa.sql.table import Table, ColumnDef, ColumnStats, ForeignKeyDef
```

ColumnDef

```python
@dataclass
class ColumnDef:
    name: str
    type_name: str
    python_type: type
    primary_key: bool = False
    not_null: bool = False
    auto_increment: bool = False
    default: Any = None
    vector_dimensions: int | None = None
    unique: bool = False
    numeric_precision: int | None = None
    numeric_scale: int | None = None
```

ColumnStats

Column-level statistics collected by `ANALYZE`:

```python
@dataclass
class ColumnStats:
    distinct_count: int = 0
    null_count: int = 0
    min_value: Any = None
    max_value: Any = None
    row_count: int = 0
    histogram: list[Any] = []
    mcv_values: list[Any] = []
    mcv_frequencies: list[float] = []
```

ForeignKeyDef

```python
@dataclass
class ForeignKeyDef:
    column: str
    ref_table: str
    ref_column: str
```

---

14. Complete Example

```python
from uqa.engine import Engine
from uqa.core.types import Vertex, Edge, Equals, GreaterThan
import numpy as np

# Create engine with persistence
engine = Engine(db_path="research.db")

# --- SQL: Create tables and insert data ---
engine.sql("""
```

```
    CREATE TABLE papers (
        id SERIAL PRIMARY KEY,
        title TEXT,
        abstract TEXT,
        year INTEGER,
        embedding VECTOR(64)
    )
""")

engine.sql("""
    INSERT INTO papers (title, abstract, year)
    VALUES ('Attention Is All You Need', 'The transformer architecture...', 2017)
""")

# --- SQL: Full-text search with BM25 ---
result = engine.sql("""
    SELECT title, _score FROM papers
    WHERE text_match(title, 'transformer attention')
    ORDER BY _score DESC
    LIMIT 10
""")

# --- SQL: Hybrid search with fusion ---
result = engine.sql("""
    SELECT title, _score FROM papers
    WHERE fuse_log_odds(
        text_match(title, 'attention'),
        knn_match(embedding, $1, 5)
    )
    ORDER BY _score DESC
""", [np.random.randn(64).tolist()])

# --- Fluent API: Term search + BM25 scoring ---
results = (
    engine.query("papers")
    .term("attention", field="abstract")
    .score_bm25("attention")
    .execute()
)
for entry in results:
    print(f"  [{entry.doc_id}] score={entry.payload.score:.4f}")

# --- Fluent API: KNN + filter ---
results = (
    engine.query("papers")
    .knn(np.random.randn(64).astype(np.float32), k=5)
    .filter("year", GreaterThan(2020))
    .execute()
)

# --- Fluent API: Graph traversal ---
engine.add_graph_vertex(Vertex(1, "Paper", {"title": "Attention"}), "papers")
engine.add_graph_vertex(Vertex(2, "Paper", {"title": "BERT"}), "papers")
```

```python
engine.add_graph_edge(Edge(1, 1, 2, "cites", {}), "papers")

cited = engine.query("papers").traverse(1, "cites", max_hops=2).execute()

# --- Cypher integration ---
result = engine.sql("""
    SELECT * FROM cypher('
        MATCH (p:Paper)-[:cites]->(cited:Paper)
        RETURN cited.title AS title
    ')
""")

# --- Export to Arrow/Parquet ---
arrow_table = result.to_arrow()
result.to_parquet("citations.parquet")

engine.close()
```